Lecture 28.03: Entropy as an Impurity Metric — Shannon Information Theory

📌 Executive Summary

Information-theoretic formulation of Shannon Entropy, bits of surprise, self-information, properties of the entropy curve, and its role as a node impurity measure in ID3 and C4.5.

🏛️ Theoretical Formulation & Mathematical Architecture

💡 Shannon Information Theory Foundations

In 1948, Claude Shannon established mathematical information theory by defining the **Self-Information** (or surprisal) of observing an event with probability $P(E)$:

$$I(E) = -\log_2 P(E) = \log_2 \left(\frac{1}{P(E)} \right)$$

The base-2 logarithm quantifies information in **bits**:

- An event with $P(E) = 1.0$ yields $I(E) = 0$ bits (no surprise).
- An event with $P(E) = 0.5$ yields $I(E) = 1$ bit (equivalent to a fair coin flip).

📐 Shannon Entropy Formulation for Node Impurity

For a dataset partition S containing samples across K discrete classes with empirical proportions $p_1, p_2, \ldots, p_K$ (where $\sum_{k=1}^{K} p_k = 1$ and $0 \log_2 0 \equiv 0$):

$$\boxed{H(S) = -\sum_{k=1}^{K} p_k \log_2(p_k)}$$

Mathematical Invariants of Entropy:

1. **Non-Negativity**: $H(S) \geq 0$ for all valid probability vectors.
2. **Absolute Purity (Minimum)**:

$$H(S) = 0 \iff \exists k ext such that p_k = 1 ext and p_{j \neq k} = 0$$

3. **Maximum Disorder (Uniform Uncertainty)**:

$$H(S) = \log_2(K) \iff p_1 = p_2 = \cdots = p_K = \frac{1}{K}$$

For a binary classification problem ($K = 2$ with positive class proportion p):

$$H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$$

The function achieves its global maximum $H(0.5) = 1.0$ bit at equal mixture.

💻 Vectorized Python Implementation

```
import numpy as np

def calculate_entropy(labels):
    _, counts = np.unique(labels, return_counts=True)
    probs = counts / counts.sum()
    # Zero probability safe mask
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))

# Test
pure_labels = np.array([1, 1, 1, 1, 1])
mixed_labels = np.array([0, 1, 0, 1, 0, 1])

print("Pure Node Entropy: ", calculate_entropy(pure_labels))
print("50-50 Node Entropy:", calculate_entropy(mixed_labels))
assert calculate_entropy(pure_labels) == 0.0
assert calculate_entropy(mixed_labels) == 1.0
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.

2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.